

Настройка модели

"snake case", множественное имя класса будет использоваться в качестве имени таблицы

По умолчанию в качестве имени для таблицы будет использоваться имя класса в "snake case" множественном числе.

```
// Задать своё имя таблицы (в классе модели):
protected $table = 'my_table';

// Свой первичный ключ
protected $primaryKey = 'flight_id';

// использовать неинкрементный или нечисловой первичный ключ
public $incrementing = false;

// Если первичный ключ должен быть строкой
protected $keyType = 'string';

// не использовать поля "created_at", "updated_at"
public $timestamps = false;

// формат даты в БД и при сериализации
protected $dateFormat = 'U';

// свои имена для "created_at", "updated_at"
const CREATED_AT = 'creation_date';
const UPDATED_AT = 'updated_date';

// подключение к бд
protected $connection = 'sqlite';

// значения по умолчанию
protected $attributes = [
    'delayed' => false,
];
```

Получение моделей

```
$flights = Flight::all();
$flight = Flight::where('number', 'FR 900')->first();

// обновить модель
$freshFlight = $flight->fresh();

// обновить модель + связи
$flight->refresh();

// использоваться для удаления моделей из коллекции
$flights = $flights->reject(function ($flight) {
    return $flight->cancelled;
});
```

```

});

// Фрагментация загрузки данных
Flight::chunk(200, function ($flights) {
    foreach ($flights as $flight) {
        //
    }
});

// используй "chunkById" для "where"
Flight::where('departed', true)
    ->chunkById(200, function ($flights) {
        $flights->each->update(['departed' => false]);
    }, $column = 'id');

// Фрагментация с использованием ленивых коллекций
foreach (Flight::lazy() as $flight) {
    //
}

// используй "lazyById" или "lazyByIdDesc" для "where"
Flight::where('departed', true)
    ->lazyById(200, $column = 'id')
    ->each->update(['departed' => false]);

// "cursor" содержит одну модель Eloquent в памяти одновременно (не может использовать отношения загрузки)
foreach (Flight::where('destination', 'Zurich')->cursor() as $flight) {
    //
}

// один запрос из таблиц "destinations" и "flights"
return Destination::addSelect(['last_flight' => Flight::select('name')
    ->whereColumn('destination_id', 'destinations.id')
    ->orderByDesc('arrived_at')
    ->limit(1)
])->get();

$flight = Flight::find(1);
$flight = Flight::where('active', 1)->first();
$flight = Flight::firstWhere('active', 1);

// найти или выполнить другое действие
$flight = Flight::findOr(1, function () {
    // ...
});
$flight = Flight::where('legs', '>', 3)->firstOr(function () {
    // ...
});

```

```
// создать исключение, если запись не найдена
$flight = Flight::findOrFail(1);
$flight = Flight::where('legs', '>', 3)->firstOrFail();

// получить или сохранить в БД
$flight = Flight::firstOrCreate(['name' => 'London to Paris']);
// получить или создать модель, но не сохранять в БД
$flight = Flight::firstOrNew(['name' => 'London to Paris']);

// Агрегатные функции
$flight = Flight::where('active', 1)->count();
$flight = Flight::where('active', 1)->max('price');

// сохранение
$flight = new Flight;
$flight->name = 'London to Paris';
$flight->save();
// или
$flight = Flight::create(['name' => 'London to Paris']);

// обновление
$flight = Flight::find(1);
$flight->name = 'Paris to Londone';
$flight->save();
// или
Flight::where(['active', 1])
    ->update(['delayed' => 1]);

$user->name = 'Vasia';
$user->isDirty();    // изменена модель? true
$user->isClean();    // не изменена модель? false
$user->save();
$user->wasChanged(); // изменена при сохранении? true

$flight->fill(['name' => 'Amsterdam to Frankfurt']);

$flight = Flight::updateOrCreate(
    ['departure' => 'Oakland']
);

Flight::upsert([
    ['departure' => 'Oakland', 'destination' => 'San Diego', 'price' => 99],
    ['departure' => 'Chicago', 'destination' => 'New York', 'price' => 150]
], ['departure', 'destination'], ['price']);
```